Assessment 2

Group 10 Members:
Ben Walker, Josh White, Chris Williams, Gavindu Tissera, Nana Twum Oviarobo, Tamerlan Urazbayev, Tymur Topala

Continuous Integration Report

As our codebase is located on github, we picked github actions as our main tool for continuous integration. We came to this decision as it integrates very well with the github workflow, allowing us to validate code automatically at multiple stages of development. Github also has a generous plan for students, allowing for three thousand minutes per month of runner time. They also have the marketplace, where there are a multitude of open source, pre-made actions to help with development, allowing us to spend less time developing our own solutions, instead focusing more time on the codebase and documentation.

To facilitate effective use of continuous integration techniques we made use of gitignore files in the repository to remove any unnecessary files, reducing the size of the remote repository. This is important, not just for maintainability, but also to reduce the time it takes to clone the repository. This helps to reduce the time that is used when running the workflows as less time is spent on the cloning stage. To further reduce the time, we use caching. By storing the last used, and, most likely still up to date versions of the software used in the workflows, we save time as we are not constantly re-downloading the same versions of software.

Once the tests have run, to keep the codebase clean, we don't merge anything that fails the tests, instead focusing on fixing the issue as a priority. This reduces the chance that a bug makes it into production code. Each PR has a coverage report showing what percentage of the code is covered by tests. Alongside automated tests, everytime we push new code to the main branch use the gradle dependency submission to keep track of all our external dependencies. This allows us to keep track of what code we are using, and get alerted about any potential vulnerabilities. By performing scans regularly, we can pick up on security vulnerabilities in the code before they become an issue.

We have several workflows set to trigger when certain events happen in the repository. To facilitate the easy tracking of change to the code base, especially between versions of code, we used a changelog, stored in the repository. This not only shows when, and what was edited, but also allows for generation of release notes when we release new stable builds. As such it is paramount that it is updated as new code is added to the project. On each pull request a short action is run, using a pre-made changelog checking script, to easily notify the maintainers that an update has been omitted. Should a changelog entry not be required, the workflow can be silenced by using the relevant label from GitHub.

While perhaps not orthodox, we have the website and the code for the game under the same repository. We felt this was justified as both are very similar in use, and it allowed easier access when passing files between the two. We are using GitHub pages to host our webpage, deployed from the "website" branch. To allow for ease of development all pull requests regarding the website are sent to main. This removes the chance that changes are made on the wrong branch. Once the changes are on main they are updated over to the website using a workflow. The website branch is protected from being pushed to by any human account, so that no accidental or malicious changes go unnoticed. To authorise the workflow to update the changes we make use of a custom GitHub app, in our case named "longboi-bot". We can give this app elevated permissions within the repository to allow it to bypass branch protection rules. The relevant access token is stored in the repository secrets, meaning that it is accessible within the workflows, but not to anyone else.

In the project we have 3 main pipelines involved in the upkeep and development of the project. All of our workflows run on linux, using the ubuntu-latest tag, which at the time of writing is version 22, although shortly to become version 24. We have checked that this brings no breaking changes to our scripts, and determined it safe to remain on the latest version. The 8 yaml files stored on the repository are split into 3 uses, maintainability, consistency, and deployment. This report will cover the 5 related to the continuous integration pipelines. The other 3 are basic implementations of continuous deployment.

When a new pull request targeting main is created the build script is run, responsible for running all unit tests, and uploading an artifact to GitHub. It runs whenever code in the game folder is changed, and can be manually triggered on any branch if required. It uses gradle to build the jar, before marking it as executable and uploading it as an artifact. The workflow also uploads the JaCoCo test report for reference.

We use a changelog file to keep track of changes between versions of the code. To ensure that it's updated every time we make use of tarides' changelog check action. It gets run anytime a pull request gets made, and targets the file at .github/CHANGELOG.md to check if it's been updated.

Once we are happy with the changes the pull request can be merged. To make sure everything stays up to date, we have 3 actions that run when new code is pushed to the main branch. The first one updates the test coverage report, taking into account the most recent changes. It's run only if changes are made to the game code. As our main branch is protected against anyone pushing code for safety reasons we make use of a custom GitHub app, called "longboi-bot" to manage permissions. It has write access to the protected branches, allowing the workflow to update the required files safely. In our update script we create a token for the bot, using that to checkout the repository with elevated permissions. The script then cleans the work area and builds the project. It then copies the test report folder to the website and commits the changes.

To keep the website up to date with changes on main, any time a push is made that updates anything in the website folder those changes need to be pushed to the website branch to be deployed. The script uses the same method as before to manage elevated permissions. When the repository is checked out using the action, we specify that it's checked out into a folder called 'root' one level down. The script then copies the website folder up one level, before switching to the website branch. To allow for the deletion of files we remove everything in the working directory before copying in the website folder. The changes are committed and force pushed to the branch. GitHub pages then runs its own deploy script to build and upload the page.

To make sure the dependency graph is always up to date with main, any time it gets pushed to we use the gradle actions to submit a dependency graph.